K. R. Mangalam University (KRMU)

School of Engineering & Technology

CAPSTONE PROJECT – PHASE II REPORT

TaskFlow

Task & Productivity Management Platform

Your day, in order.

Course	ENSP465 – New Age Programming Languages Lab
Project Title	TaskFlow – Task & Productivity Management Platform
Report	Capstone Project – Phase II Documentation
Student Name	Sumukhi Pandey
Roll Number	2301010184
Program / Year	B.Tech, 4th Year
Semester / Section	VII / C
Academic Year	2026–27
Faculty Name	Dr. Hemraj Kumawat
School	School of Engineering & Technology, KRMU
GitHub Repository	https://github.com/Sumukhi90/TaskFlow
Submission Date	5 November 2026

1. Submission Summary

This document is the Phase II submission for the ENSP465 capstone project. It covers the three items the university asks for: the repository link, a short written documentation of the project, and screenshots of the working user interface.

Requirement	Where it is covered
GitHub repository link	https://github.com/Sumukhi90/TaskFlow
Short documentation	Sections 2 to 7 (overview, objectives, features, technologies, architecture, implementation) and Sections 9 to 12
UI screenshots	Section 8, Figures 1 to 6
Submission date	5 November 2026
Deadline	10 November 2026

2. Project Overview

TaskFlow is a web-based task and productivity management platform. A signed-in user keeps all of their tasks and projects in one place, sees what is due, and reviews how their work is going through a dashboard, a calendar and analytics.

The problem. Study deadlines, internship work and personal errands usually end up scattered across notes, chats and memory. Items get missed because nothing shows which task is overdue, which is most important, or how much of a project is finished.

Why it helps. TaskFlow gathers every task with its due date, status, priority, project and tags, and highlights overdue work. Each user only sees their own data, and everything is stored in a database, so it is still there the next time they sign in.

Intended users. Students and early-career professionals who handle several parallel commitments, for example coursework, exam preparation and an internship.

3. Project Objectives

- **Centralised task management** – create, edit, complete and delete tasks from a single interface.
- **Deadline and priority tracking** – every task carries a due date, a status and a priority, with overdue work flagged.
- **Project organisation** – group tasks under projects and follow progress and task counts per project.
- **Authentication and data separation** – users sign in, and tasks and projects belong to the user who created them.
- **Productivity insights** – dashboard figures, a monthly calendar, analytics charts and an activity history.
- **Persistent storage** – keep data in an SQLite database instead of in the browser or in memory.
- **A responsive, user-friendly interface** – clear screens, loading, empty and retry states, and a light and dark theme.

4. Key Features

#	Feature	What it does
1	Login and logout	Users sign in with email and password and can sign out from the sidebar.
2	Database-backed sessions	Sessions are stored in the database, not only in the browser.
3	Password hashing	Passwords are stored as PBKDF2-SHA256 hashes.
4	User-specific data	Tasks and projects are tied to the signed-in user.
5	Task management	Create, edit, delete and mark tasks complete.
6	Tags and priorities	Each task has a priority and comma-separated tags.
7	Search, filter, sort	Search text, filter by status, priority, project and due date, and sort the list.
8	Dashboard statistics	Totals for to do, in progress, completed and overdue tasks, with a completion percentage.
9	Project progress	Progress and task counts for each project.
10	Monthly calendar	Tasks placed on their due dates, with month navigation.
11	Analytics charts	Charts that summarise completed work.
12	Activity history	A timeline of task and project changes.
13	Profile and theme	Profile page and settings, including light and dark themes.
14	Validation and errors	Requests are validated and invalid input returns an error.
15	Loading, empty, retry states	The interface handles waiting, no-data and failed-request cases.
16	SQLite schema migration	The schema is upgraded additively so existing task data is kept.

5. Technologies Used

Technology	Purpose in TaskFlow
Go	Backend language; the server is written as `package main` in the `backend` folder.
Go net/http	HTTP server, request routing and handlers for the REST API.
encoding/json	Encoding and decoding of JSON request and response bodies.
database/sql	Standard Go interface for database access; the server holds a `*sql.DB`.
SQLite	Embedded relational database that stores users, sessions, tasks and projects (file `taskflow.db`).
modernc.org/sqlite	Pure-Go SQLite driver, registered with a blank import in `main.go`.
regexp, strconv, strings, time	Go standard-library packages used in `tasks.go` for ID checks, parsing, text handling and dates.

Technology	Purpose in TaskFlow
React	Frontend user interface (screens and components).
Vite	Frontend development server and build tool; the app runs on port 5173.
JavaScript	Frontend logic, including the API service layer.
HTML / CSS	Page structure and styling, responsive layout and themes.
PowerShell	`start.ps1` script that starts the project on Windows.
Go testing and httptest	Automated tests for the API in `main_test.go`.

6. System Architecture

TaskFlow is a three-tier application: a React/Vite frontend talks to a Go REST API, and the API reads and writes an SQLite database.

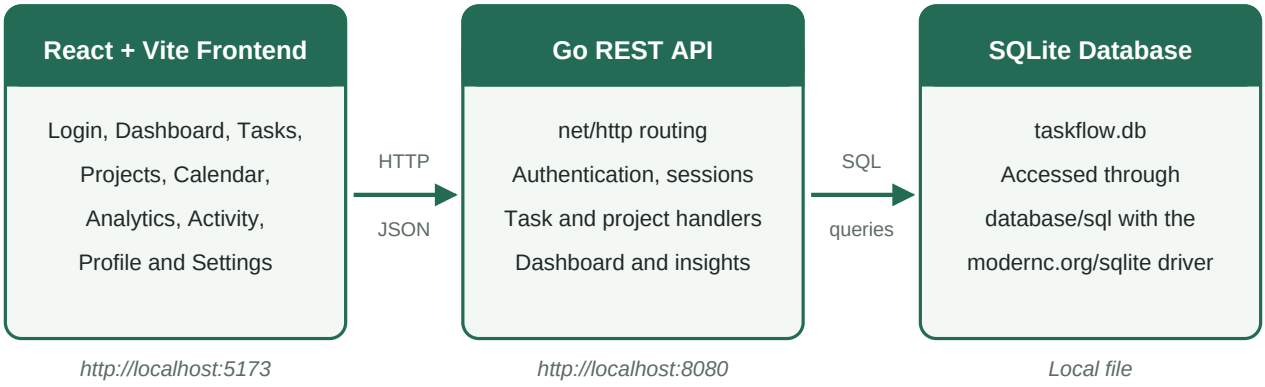


Figure A. *TaskFlow architecture: React/Vite frontend → Go REST API → SQLite database.*

Layer	Responsibility
Frontend	Renders the screens, collects user input, keeps interface state and calls the API.
API / backend	Routes requests, validates input, applies the business rules and runs the task, project and insight queries.
Authentication and sessions	Checks credentials against hashed passwords, creates and verifies sessions, and identifies the user for each request.
Database	Stores all persistent data and keeps each user’s records separate.
API communication	JSON over HTTP between the frontend and the backend, with credentials (session cookie) sent on each request.

7. Implementation Details

7.1 Backend

The backend uses Go's standard `net/http` package. A `Server` struct holds the database handle, and each API area has its own handler methods on that struct. A handler such as `tasks` looks at the HTTP method and passes GET and POST requests to separate functions (`listTasks`, `createTask`); any other method gets a method-not-allowed response. Task IDs are checked against a regular expression before use. The database file location is read from the `TASKFLOW_DB` environment variable.

7.2 Authentication

- Passwords are hashed with PBKDF2-SHA256.
- Sessions are stored in the database and identified by random tokens.
- The token is sent as a cookie marked HttpOnly and SameSite.
- Sessions expire after seven days.
- The signed-in user's ID is carried through the request context (`userIDKey` in `main.go`) so handlers only touch that user's records.

7.3 Database

SQLite holds the application data in `taskflow.db`. Tasks and projects are linked to users, and tasks can be linked to a project, so every query is limited to the signed-in user. When the schema changes, the migration only adds to it, which keeps tasks that already exist.

7.4 Frontend

The frontend is a React application built with Vite. It has screens for login, Dashboard, Tasks, Projects, Calendar, Analytics, Activity, Profile and Settings, plus a modal form for creating and editing tasks. A service layer in JavaScript makes the API calls with credentials included so the session cookie is sent. The layout is responsive, a light and dark theme is available in Settings, and screens show loading, empty and retry states.

7.5 Testing

The backend tests are in `main_test.go` and use Go's `testing` and `httptest` packages with an in-memory SQLite database. They cover authentication and API behaviour, task validation and CRUD, project operations, insights and profile endpoints, and that a schema migration preserves existing task data. Run `go test` in the `backend` folder to execute them.

7.6 Backend source files

File	Role
main.go	Server struct, start-up and route registration
database.go	Database initialisation and schema migration
auth.go	Password hashing and session handling
tasks.go	Task endpoints and validation
projects.go	Project endpoints
insights.go	Dashboard, calendar and analytics queries
models.go	Request, response and data structures
main_test.go	Automated tests

8. Application Screenshots and Feature Demonstration

The screenshots below were taken from the running application. They are shown unedited.

8.1 Login

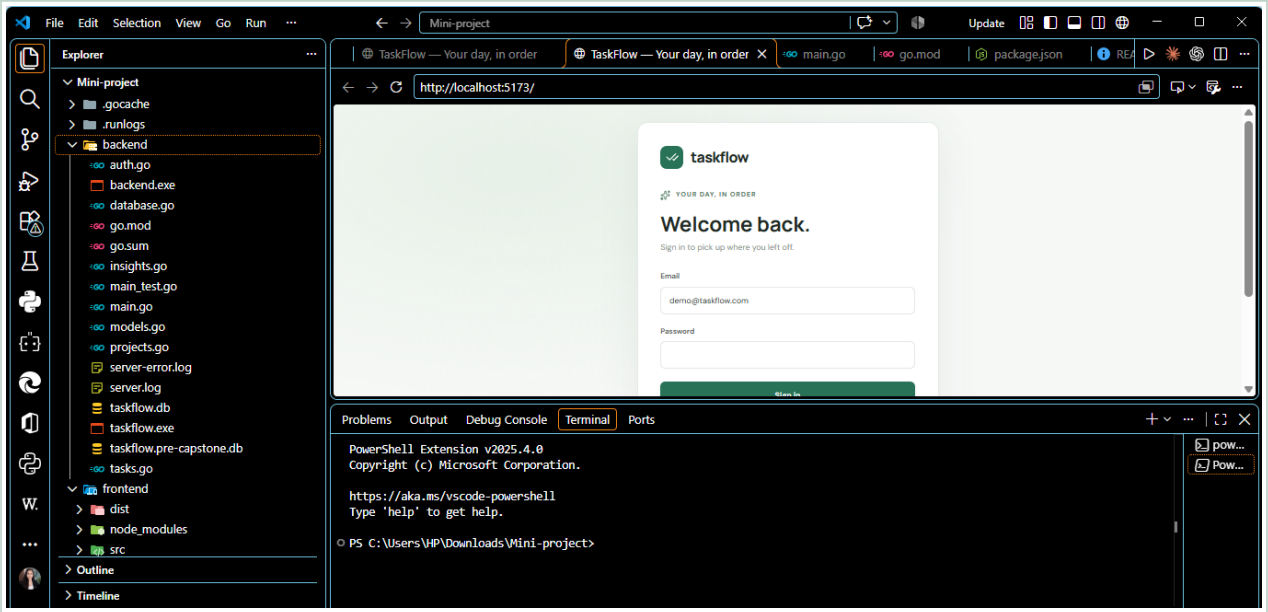


Figure 1. *Login page served at http://localhost:5173, shown in the editor's browser pane next to the project folders (backend and frontend).*

The sign-in form asks for an email and password and leads into the user's own workspace. The explorer on the left shows the backend Go files and the frontend folder.

8.2 Dashboard

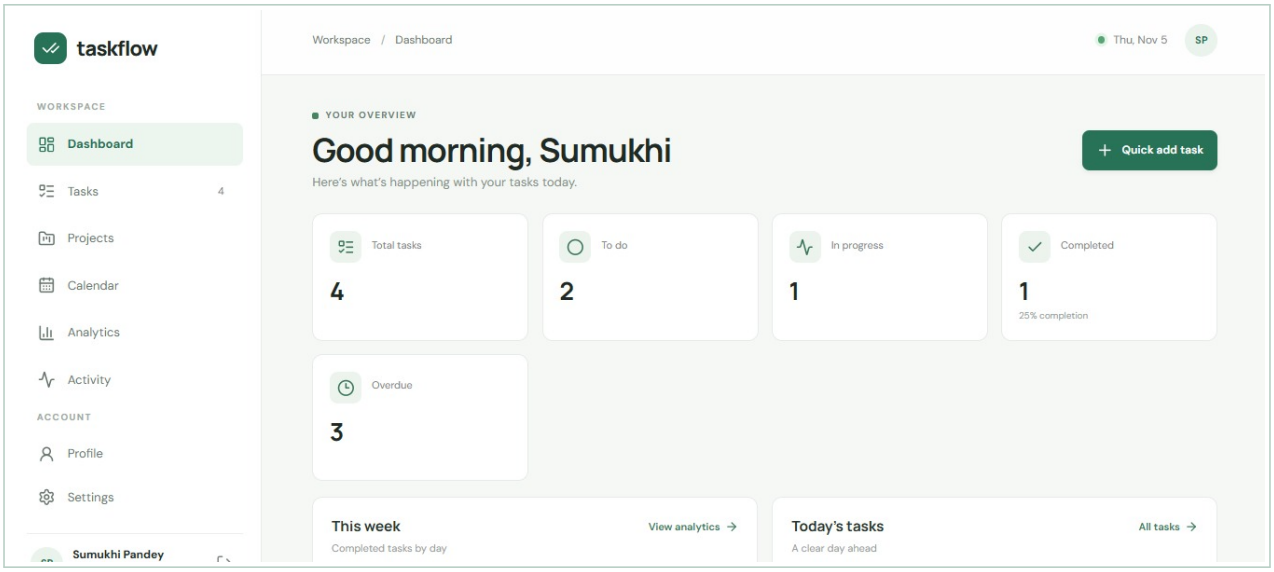


Figure 2. *Dashboard with the sidebar navigation and summary cards.*

The dashboard greets the user and shows counts for total, to do, in progress, completed (with a completion percentage) and overdue tasks. The sidebar links to Tasks, Projects, Calendar, Analytics, Activity, Profile and Settings, and a *Quick add task* button is always available.

8.3 Task management, search and filtering

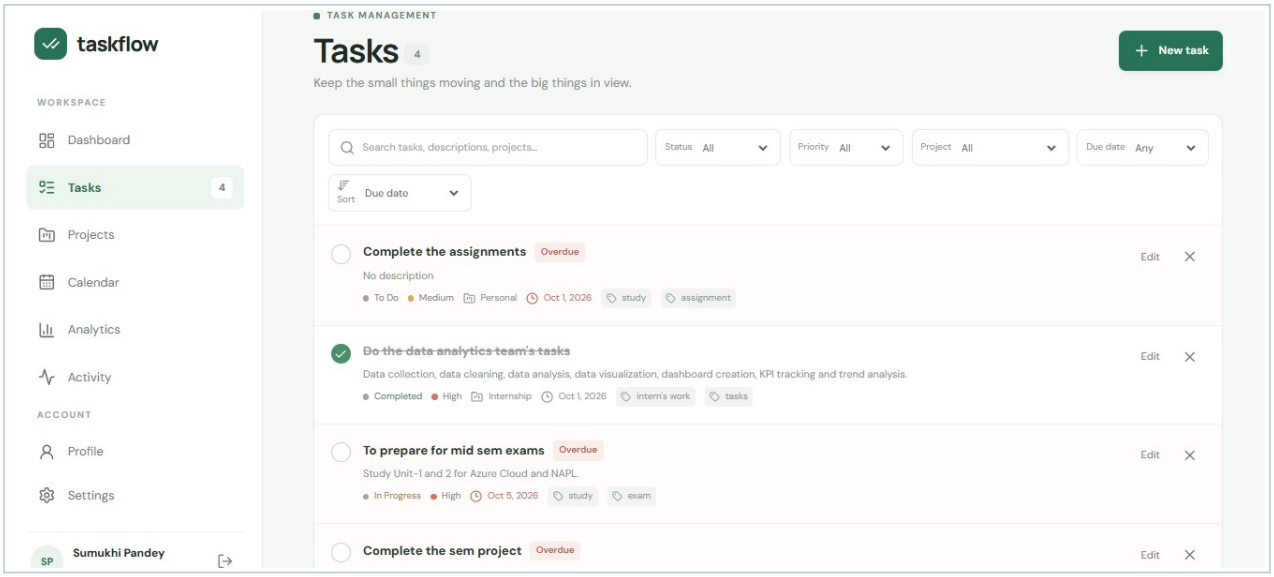


Figure 3. Task list with search box, filters and sorting.

Tasks can be searched by text and filtered by status, priority, project and due date, then sorted. Each row shows status, priority, project, due date and tags, flags overdue tasks, strikes through completed ones, and offers Edit and delete actions.

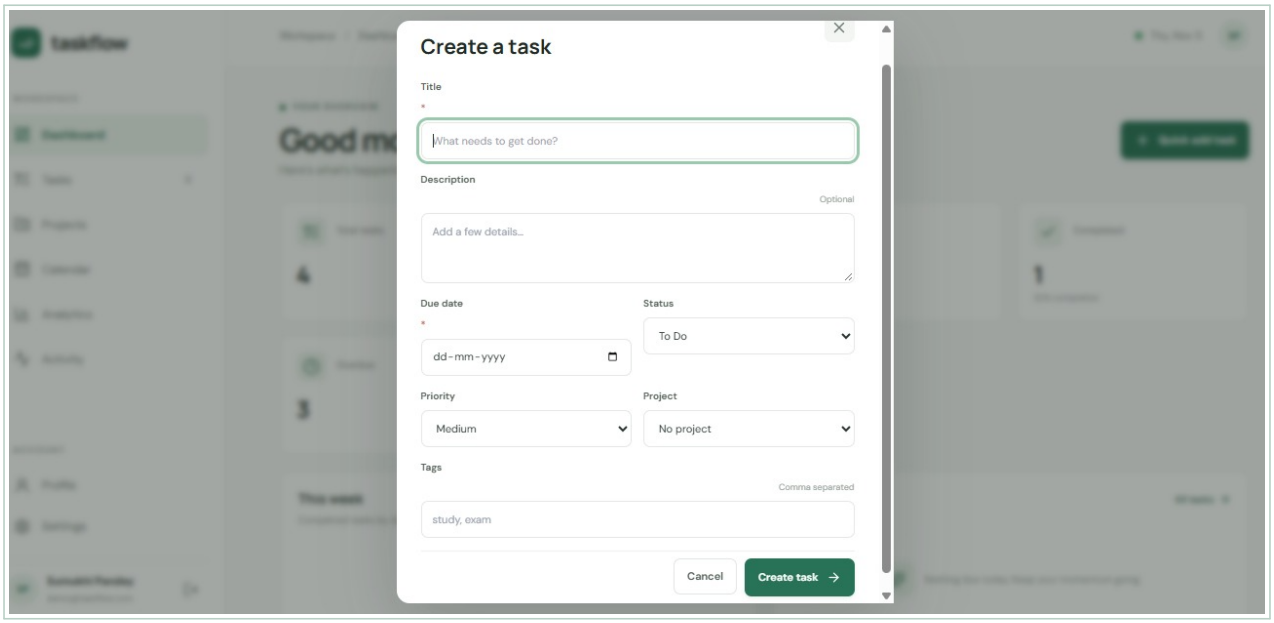


Figure 4. Create a task dialog.

The form takes a title and due date (both required), an optional description, a status, a priority, a project and comma-separated tags.

8.4 Calendar

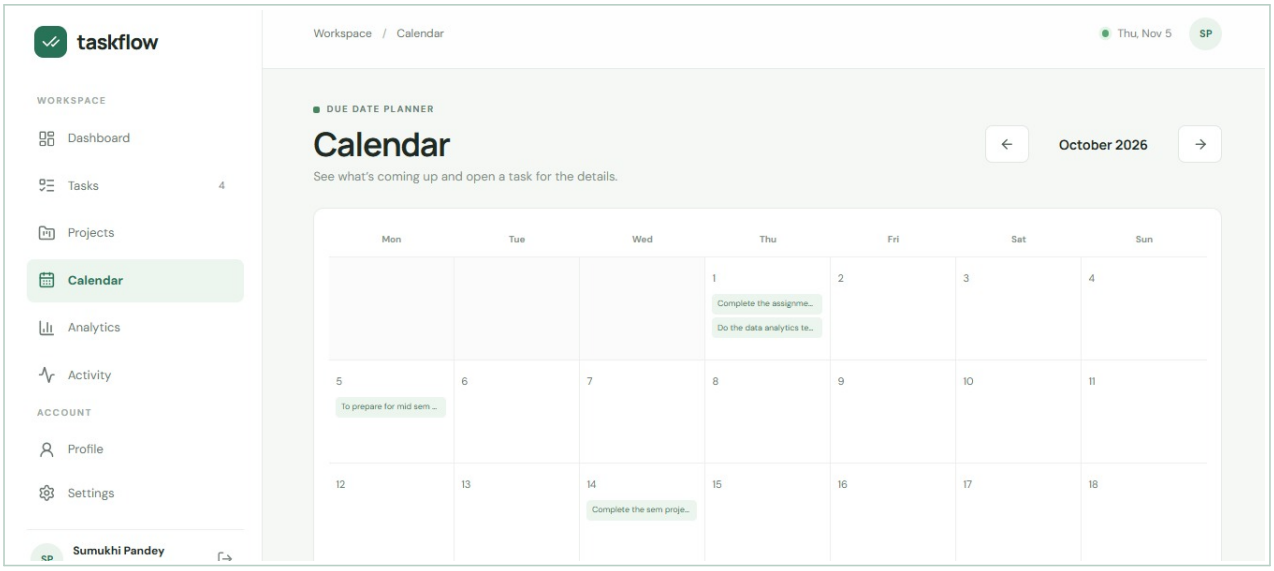


Figure 5. *Monthly calendar view for October 2026.*

Tasks appear on their due dates, and the arrow buttons move between months.

8.5 Activity history

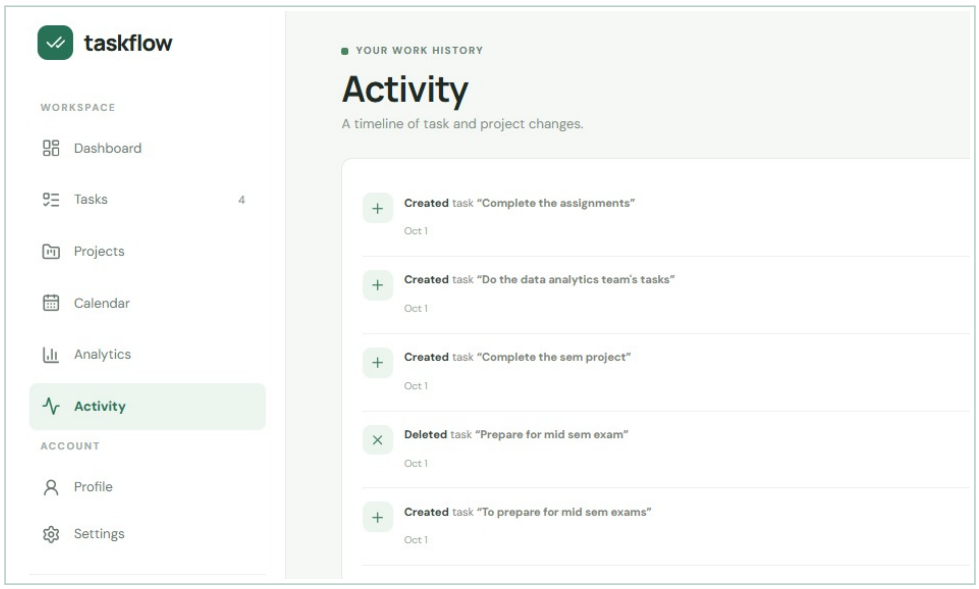


Figure 6. *Activity timeline.*

The timeline records task and project changes such as created and deleted tasks, with the date of each event.

9. Important Code Snapshots and Implementation Evidence

The excerpts below are taken from the project files in the `backend` folder and are shortened to the relevant lines.

9.1 main.go – server type, context key and start-up

```
import (
    "context"
    "database/sql"
    "encoding/json"
    "errors"
    "io"
    "log"
    "net/http"
    "os"
    "time"

    _ "modernc.org/sqlite"
)

type Server struct{ db *sql.DB }

type contextKey string

const userIDKey contextKey = "taskflow-user-id"

func main() {
    dbPath := os.Getenv("TASKFLOW_DB")
    if dbPath == "" {
```

The `Server` struct wraps a `*sql.DB`, and the SQLite driver is registered through a blank import. A typed context key is defined for the signed-in user's ID, and the database path is read from an environment variable. *Concepts:* structs, package imports, custom types, environment configuration.

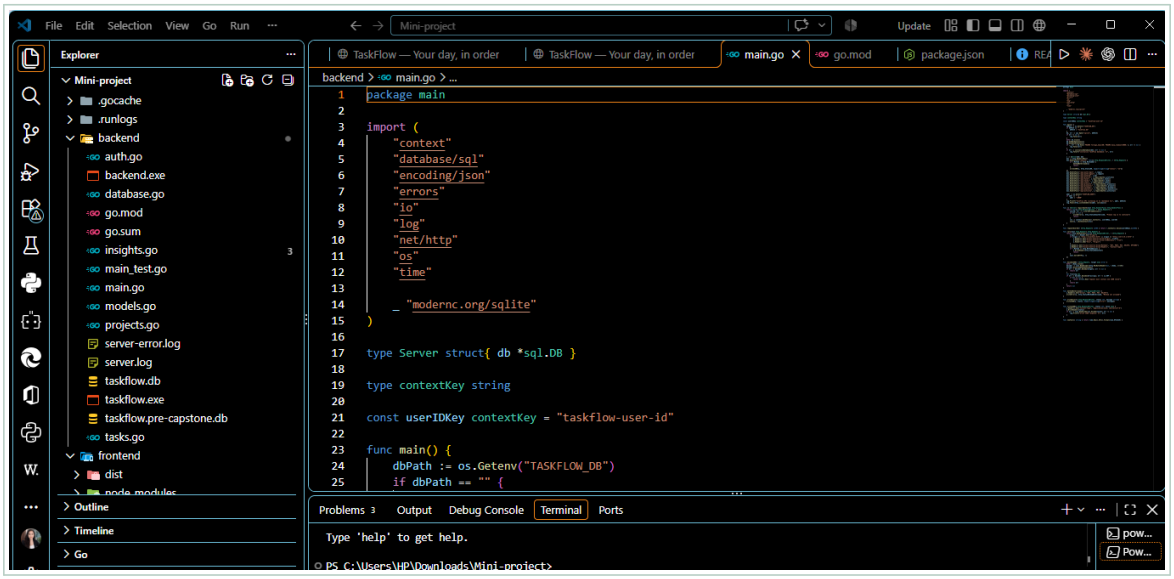


Figure 7. *main.go open in the editor (backend/main.go).*

9.2 tasks.go – task ID validation and method dispatch

```
var taskIDPattern = regexp.MustCompile(`^[1-9][0-9]*$`)

func (s *Server) tasks(w http.ResponseWriter, r *http.Request) {
    switch r.Method {
    case http.MethodGet:
        s.listTasks(w, r)
    case http.MethodPost:
        s.createTask(w, r)
    default:
        methodNotAllowed(w)
    }
```

A compiled regular expression accepts only positive whole numbers without leading zeros as task IDs. The `tasks` handler is a method on `Server` and uses a `switch` on the HTTP method: GET lists tasks, POST creates one, and anything else is rejected. *Concepts:* methods with pointer receivers, `net/http` handlers, regular expressions, switch statements.

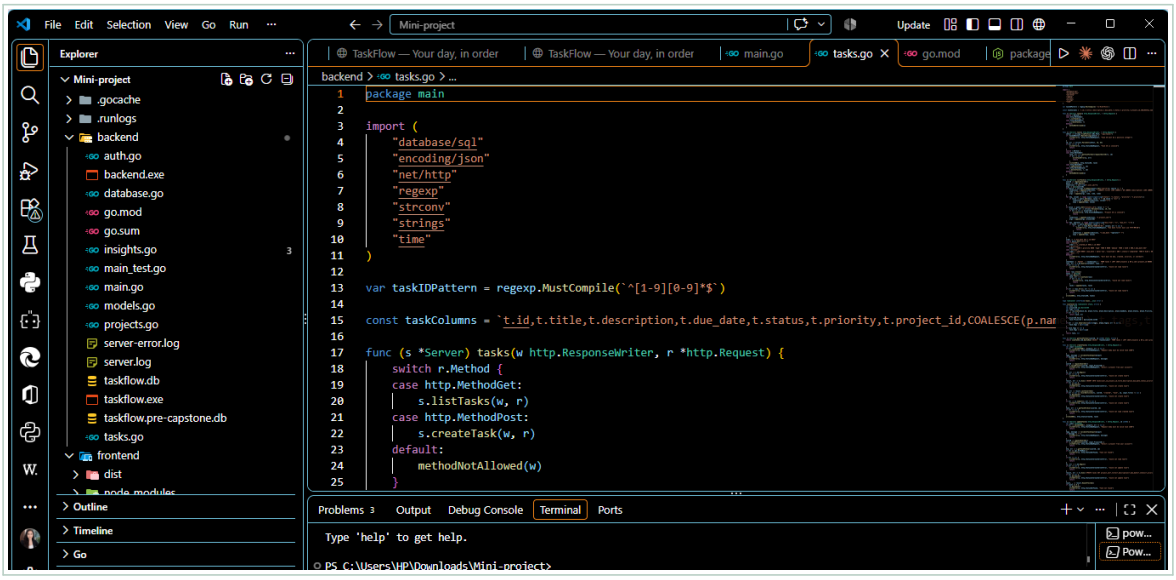


Figure 8. *tasks.go open in the editor (backend/tasks.go).*

10. Project Execution (Windows)

1. Open the project directory (`Mini-project`) in PowerShell.
2. Run `start.ps1`.
3. Open `http://localhost:5173` in a browser to use TaskFlow.

The React/Vite frontend is served on port 5173. The Go API runs on port 8080 and the frontend sends its requests to it.

11. Scope and Future Enhancements

TaskFlow is built as a focused, local task and productivity application, and its current scope is deliberately clear. The core workflow is complete and working, and the choices below leave room to grow.

- **Easy to run and explore.** As a local application, TaskFlow starts with a single script and needs no external services.
- **Ready for account features.** The hashed-password and session foundation is in place, so self-service registration, password reset and email verification are natural next steps.
- **Simple, dependable storage.** SQLite keeps setup light for local, single-instance use, and the use of `database/sql` keeps the door open to a server database as usage grows.
- **Path to deployment.** Adding HTTPS hosting is the step that would take TaskFlow from a local demo to a production deployment.
- **Convenient sign-in.** Sessions stay active for seven days, so users are not asked to sign in repeatedly.

12. Conclusion

TaskFlow brings tasks, projects, deadlines and progress into one signed-in workspace. The React interface handles what the user sees and does, the Go API enforces validation, authentication and user-level separation, and SQLite keeps the data between sessions.

Technically, the project shows a complete full-stack flow: a Go backend built on the standard library and `database/sql`, hashed passwords with database-backed sessions, an additive schema migration, a tested API, and a responsive frontend with a dashboard, calendar, analytics and activity history. As a productivity tool it helps a user see what is overdue, what is next, and how their projects are moving.